



Week 4 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 27, 2021 2:41 PM
🔗 Materials	
# Module #	24
▼ Type	Lecture
☰ Week #	4

Mechanisms for Persistent Storage

In-memory data structures

```
names = ['Alice', 'Bob', 'Charlie']
courses = ['Introduction to EE', 'Applied Mech', 'Calculus']
rels = [('Alice', 'Introduction to EE'),
        ('Bob', 'Calculus'),
        ('Alice', 'Calculus'),
        ('Charlie', 'Applied Mech')]
```

- Error prone - easy to make mistakes in entry or referencing
- Does not scale
- Duplicate names?

In-memory data structures: Keys

```
names = {0: 'Alice', 1: 'Bob', 2: 'Charlie'}
courses = {0: 'Introduction to EE', 1: 'Applied Mech', 2: 'Calculus'}
rels = [(0, 0), (1, 2), (0, 2), (2, 1)]
```

- Data entry errors less likely
- Duplicates not a problem - Unique key

Objects

```
class Student:
    id_next = 0 # Class variable
    def __init__(self, name):
        self.name = name
        self.id = Student.id_next
        Student.id_next += 1
```

- Auto-initialize ID to ensure unique
- Functions to set/get values

```
class Student:
    id_next = 0 # Class variable
    def __init__(self, name, hostel):
        self.name = name
        self.id = Student.id_next
        self.hostel = hostel
        Student.id_next += 1
```

- Add a new field to the object easily

But what about persistence?

- In-memory data structures are lost when the program is killed, server is shut down or rebooted (restarted)
- Save to disk? Structured data?
 - Python Pickle and similar modules
 - CSV - Comma Separated Values
 - TSV - Tab Separated Values
- Essentially the same as spreadsheets - limited flexibility

Spreadsheets

- Naturally represent tabular data
- Extension, adding fields are easy
- Separate sheets for relationships

Problems with spreadsheets:

- Lookups, cross-referencing harder than dedicated database
- Stored procedures - limited functionality
- Atomic operations - no clear definition

Relational Databases - SQL

Shameless plug, I do have notes for DBMS here

- From IBM ~ 1970s
- Data is stored in tabular format:
 - Columns of tables: fields (name, address, department, ...)
 - Rows of tables: individual entries (student1, student2, ...)

Unstructured Databases - NoSQL

- Easily add/change fields
- Arbitrary data
- NoSQL
 - MongoDB
 - CouchDB
 - ...

- Flexible, but potential loss of validation